Report: Detailed Analysis of Encryption and Decryption Programs

Introduction

This comprehensive report delves into the intricacies of two C programs, each designed for distinct purposes: Program 1 focuses on text encryption, while Program 2 handles decryption. By examining the specific functionalities within these programs, we aim to gain a deeper understanding of their underlying algorithms.

Program 1: Encryption Algorithm

Purpose:

The primary goal of Program 1 is to encrypt user-inputted text using a specific algorithm.

Functionality:

1)PROCESSING COMMAND BLOCKS: The program begins by identifying comment blocks, initiated)by the '/' character. It then counts the non-space characters within the comment block and displays this count on the screen.

```
if (character == '/') {
    scanf("%c", &character);

    if (character == '*') {
        /* Loop to count characters in the comment block */
        while (scanf("%c", &character) != EOF && character != '*') {
            if (character != ' ') {
                counter++;
            }
        }
    }
}
```

2)CHARACTHER ENCRYPTION: Characters outside of comment blocks undergo encryption by shifting within a predefined alphabet array, utilizing a fixed key value.

```
int i = 0;

while (i < 61) {

	if (character == alphabet[i]) {

		printf("%c", alphabet[(i + key) % 61]);

		break;

	}

	i++;

}
```

3)PRINTING TO SCREEN: The encrypted result, including characters represented by a special character encoding the count of non-space characters in comment blocks, is then printed to the screen.

```
if (character == '\n' || character == ' ') {

	printf("%c", character);

}
```

Key Value:

The key value for encryption is statically set to 3, derived from the sum of the numbers in the user's school number.

Program 2: Decryption Algorithm

Purpose:

Program 2 is designed to decrypt text previously encrypted by Program 1.

Functionality:

1)PROCESSING COMMAND BLOCKS: Similar to Program 1, the program identifies comment blocks initiated by the '/' character. It retrieves the count of characters within the comment block and prints this information to the screen.

```
if (character == yorumsat) {
    scanf("%d", &numberofchar);
    printf(" /* There are %d characters in the comment. */", numberofchar);
}
```

2)CHARACTER DECRYTİON: Encrypted characters are decrypted by reversing the shift using the fixed key value.

```
int i = 0;
while (i < 61) {
    if (character == alphabet[i]) {
        printf("%c", alphabet[(i - key + 61) % 61]);
        break;
    }
    i++;
}
```

3)PRINTING TO SCREEN: The original text, decrypted from the encrypted input, is then printed to the screen. Characters within comment blocks are represented by a special character encoding the count of non-space characters.

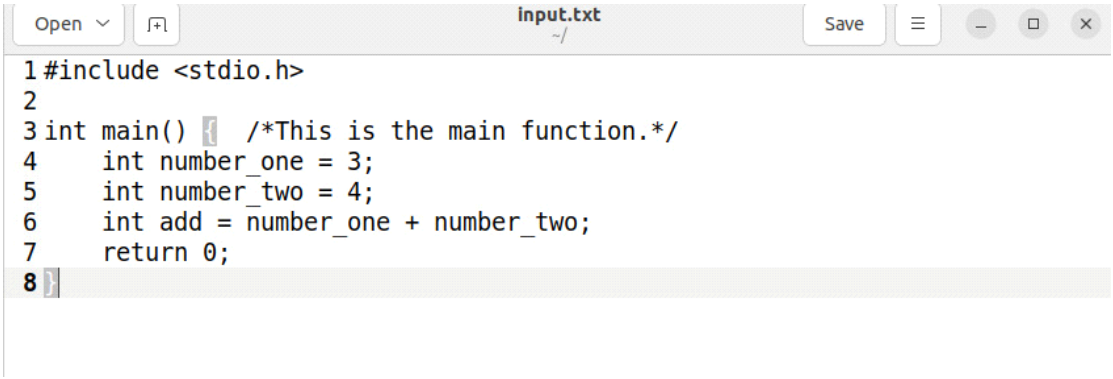
```
if (character == '\n' || character == ' ') {
    printf("%c", character);
}
```

Conclusion

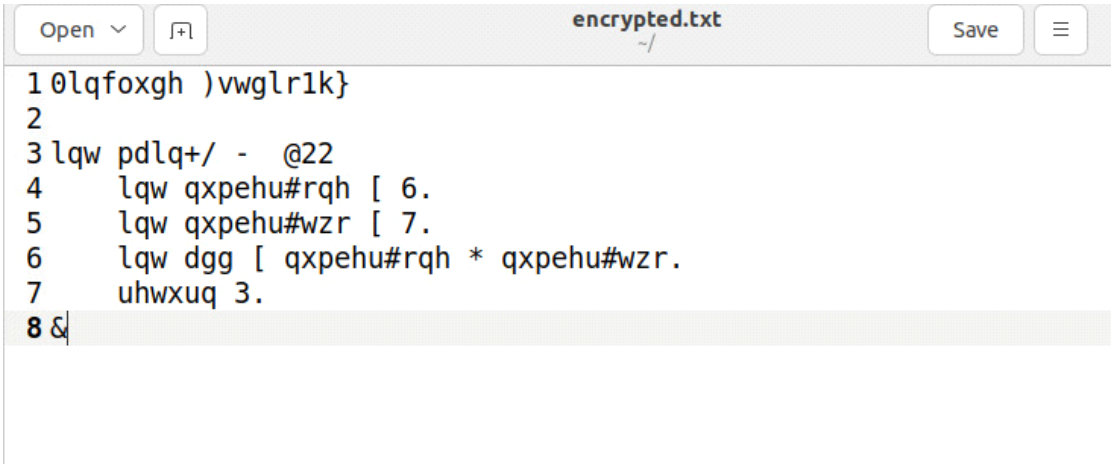
In conclusion, the provided C programs effectively demonstrate a simple yet functional encryption and decryption process. The report has outlined the specific functionalities of each program, shedding light on their individual purposes within the context of text manipulation.

OUTPUTS:

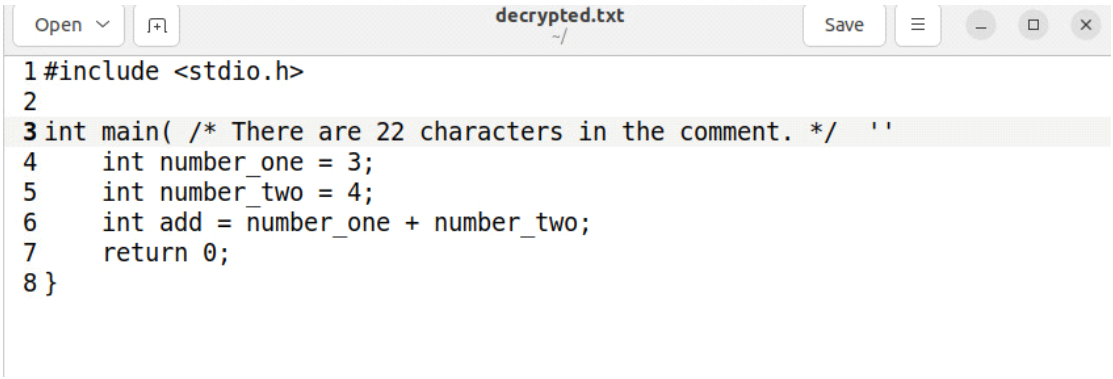
input.txt



encrypted.txt



decrypted.txt


Open
decrypted.txt
~/
Save
1 #include <stdio.h>
2
3 int main(/* There are 22 characters in the comment. */ ''
4   int number_one = 3;
5   int number_two = 4;
6   int add = number_one + number_two;
7   return 0;
8 }